VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: FCFS and SJF Scheduling algorithms

Experiment No: 1 **Date:** 23-07-2026

Aim: write a c-program for FCFS scheduling algorithm A. without arrival times, B. with arrival times

A. without arrival times

Algorithm:

1. Start.
2. Input the number of processes (n).
3. Input the Arrival Time (AT) and Burst Time (BT) for each process.
4. Initialize current_time = 0, total_wt = 0, and total_tat = 0.
5. For each process:
    - If current_time < AT, set current_time = AT.
    - Calculate CT = current_time + BT.
    - Calculate TAT = CT − AT.
    - Calculate WT = TAT − BT.
    - Update current_time = CT.
6. Add WT and TAT to their respective totals.
    - Calculate:
    - Average Waiting Time = Total WT / n
    - Average Turnaround Time = Total TAT / n
7. Display AT, BT, CT, WT, TAT, Average WT, and Average TAT.
8. Stop.

B. with arrival times

Algorithm:

1. Start.
2. Input the number of processes (n).
3. Input the Burst Time (BT) for each process.
4. Initialize WT[0] = 0, total_wt = 0, and total_tat = 0.
5. For each process:
    - Calculate WT using the previous process's burst time.
    - Calculate TAT = WT + BT.
    - Add WT and TAT to their respective totals.
6. Calculate:
    - Average Waiting Time = Total WT / n
    - Average Turnaround Time = Total TAT / n
7. Display BT, WT, TAT, Average WT, and Average TAT.
8. Stop.

A. without arrival times

Program:

```
#include <stdio.h>
int main() {
   int n, bt[20], wt[20], tat[20];
   float twt = 0, ttat = 0;
   printf("Roll no: 25071A6226\n");
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Burst Time for P%d: ", i + 1);
      scanf("%d", &bt[i]);
   }
   wt[0] = 0;
   for (int i = 0; i < n; i++) {
      if (i > 0) wt[i] = wt[i - 1] + bt[i - 1];
      tat[i] = wt[i] + bt[i];
      twt += wt[i]; ttat += tat[i];
   }
   printf("\nProcess\tBT\tWT\tTAT\n");
   for (int i = 0; i < n; i++)
      printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
   printf("\nAvg WT = %.2f | Avg TAT = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Burst Time for P1: 24
Enter Burst Time for P2: 3
Enter Burst Time for P3: 3

Process  BT       WT       TAT
P1       24       0        24
P2       3        24       27
P3       3        27       30

Avg WT = 17.00 | Avg TAT = 27.00
```

```
Roll no: 25071A6236
Enter number of processes: 4
Enter Burst Time for P1: 2
Enter Burst Time for P2: 10
Enter Burst Time for P3: 4
Enter Burst Time for P4: 7

Process  BT       WT       TAT
P1       2        0        2
P2       10       2        12
P3       4        12       16
P4       7        16       23

Avg WT = 7.50 | Avg TAT = 13.25
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Burst Time for P1: 2
Enter Burst Time for P2: 4
Enter Burst Time for P3: 6
Enter Burst Time for P4: 8

Process  BT       WT       TAT
P1       2        0        2
P2       4        2        6
P3       6        6        12
P4       8        12       20

Avg WT = 5.00 | Avg TAT = 10.00
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

B. with arrival times

Program:

```
#include <stdio.h>
int main() {
   int n, at[20], bt[20], time = 0;
   float twt = 0, ttat = 0;
   printf("Roll no: 25071A6226\n");
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]);
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      if (time < at[i]) time = at[i];
      time += bt[i];
      int tat = time - at[i], wt = tat - bt[i];
      twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], time, wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Burst Time for P1: 24
Enter Burst Time for P2: 3
Enter Burst Time for P3: 3

Process BT        WT        TAT
P1        24        0         24
P2        3         24        27
P3        3         27        30

Avg WT = 17.00 | Avg TAT = 27.00
```

```
Roll no: 25071A6236
Enter number of processes: 4
Enter Burst Time for P1: 2
Enter Burst Time for P2: 10
Enter Burst Time for P3: 4
Enter Burst Time for P4: 7

Process BT          WT          TAT
P1        2         0           2
P2        10        2           12
P3        4         12          16
P4        7         16          23

Avg WT = 7.50 | Avg TAT = 13.25
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Burst Time for P1: 2
Enter Burst Time for P2: 4
Enter Burst Time for P3: 6
Enter Burst Time for P4: 8

Process BT          WT          TAT
P1        2         0           2
P2        4         2           6
P3        6         6           12
P4        8         12          20

Avg WT = 5.00 | Avg TAT = 10.00
```

Aim: write a c-program for SJF scheduling algorithm A. Non-preemptive B. Preemptive

A. non-preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT).
3. Initialize current_time = 0 and mark all processes as unvisited.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ current_time) with the smallest Burst Time (BT).
    - If no process has arrived, increase current_time by 1.
5. Otherwise:
    - Execute the selected process.
    - Calculate CT, TAT, and WT.
    - Update current_time = CT.
    - Mark the process as completed.
6. Calculate the Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
7. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
8. Stop.

B. Preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ time) with the smallest Remaining Time (RT).
    - If no process is ready, increase time by 1.
    - Otherwise:
        - Execute the selected process for 1 time unit.
        - Decrease RT by 1 and increase time by 1.
        - If RT = 0, calculate CT, mark the process as completed, and increment completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

A. non-preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, p[10], at[10], bt[10], ct[10], tat[10], wt[10], vis[10]={0}, cur=0, done=0;
   float twt=0, ttat=0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for(int i=0; i<n; i++) { p[i]=i+1; printf("Enter Arrival Time and Burst Time for P%d: ", i+1); scanf("%d%d",
&at[i], &bt[i]); }
   while(done < n) {
      int idx = -1, min_bt = 1e9;
      for(int i=0; i<n; i++)
         if(at[i] <= cur && !vis[i] && bt[i] < min_bt) { min_bt = bt[i]; idx = i; }
      if(idx == -1) cur++;
      else {
         cur += bt[idx]; ct[idx] = cur; tat[idx] = ct[idx] - at[idx];
         wt[idx] = tat[idx] - bt[idx]; twt += wt[idx]; ttat += tat[idx]; vis[idx] = 1; done++;
      }
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for(int i=0; i<n; i++) printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], wt[i], tat[i]);
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt/n, ttat/n);
   return 0;}
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 5 6
Enter Arrival Time and Burst Time for P3: 6 2

Process  AT       BT       CT       WT       TAT
P1       0        3        3        0        3
P2       5        6        11       0        6
P3       6        2        13       5        7

Average Waiting Time = 1.67
Average Turnaround Time = 5.33
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 7
Enter Arrival Time and Burst Time for P2: 2 4
Enter Arrival Time and Burst Time for P3: 4 1
Enter Arrival Time and Burst Time for P4: 5 4

Process  AT       BT       CT       WT       TAT
P1       0        7        7        0        7
P2       2        4        12       6        10
P3       4        1        8        3        4
P4       5        4        16       7        11

Average Waiting Time = 4.00
Average Turnaround Time = 8.00
```

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process  AT       BT       CT       WT       TAT
P1       0        24       30       6        30
P2       0        3        3        0        3
P3       0        3        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

B. preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, at[10], bt[10], rt[10], ct[10], done = 0, time = 0, min_rt, idx;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];
   }
   while (done < n) {
      min_rt = 1e9; idx = -1;
      for (int i = 0; i < n; i++)
         if (at[i] <= time && rt[i] > 0 && rt[i] < min_rt) { min_rt = rt[i]; idx = i; }
      if (idx == -1) { time++; continue; }
      rt[idx]--; time++;
      if (rt[idx] == 0) { done++; ct[idx] = time; }
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i];
      twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 2

Process  AT       BT       CT       WT       TAT
P1       0        8        14       6        14
P2       1        4        7        2        6
P3       2        2        4        0        2

Average Waiting Time = 2.67
Average Turnaround Time = 7.33
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 9
Enter Arrival Time and Burst Time for P4: 3 5

Process  AT       BT       CT       WT       TAT
P1       0        8        17       9        17
P2       1        4        5        0        4
P3       2        9        26       15       24
P4       3        5        10       2        7

Average Waiting Time = 6.50
Average Turnaround Time = 13.00
```

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process  AT       BT       CT       WT       TAT
P1       0        24       30       6        30
P2       0        3        3        0        3
P3       0        3        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Name of the Experiment:

Experiment No: **Date:**

Aim: write a c-program for priority scheduling algorithm A. Non-preemptive B. Preemptive

A. non-preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), Burst Time (BT), and Priority (Pr).
3. Initialize current_time = 0 and mark all processes as unvisited.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ current_time) with the highest priority (lowest Priority value).
    - If no process is ready, increase current_time by 1.
    - Otherwise:
        - Execute the selected process.
        - Calculate CT, TAT, and WT.
        - Update current_time = CT.
        - Mark the process as completed.
5. Calculate Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
6. Display AT, BT, Priority, CT, WT, TAT, AWT, and ATAT.
7. Stop.

B. Preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), Burst Time (BT), and Priority (Pr). Set Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ time) with the highest priority (lowest Priority value).
    - If no process is ready, increase time by 1.
    - Otherwise:
        - Execute the selected process for 1 time unit.
        - Decrease RT by 1 and increase time by 1.
        - If RT = 0, calculate CT, mark the process as completed, and increment completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, Priority, CT, WT, TAT, AWT, and ATAT.
7. Stop.

A. non-preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, at[10], bt[10], pr[10], ct[10], vis[10]={0}, cur=0, done=0;
   float twt=0, ttat=0;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(int i=0; i<n; i++) {
 printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i+1); scanf("%d%d%d", &at[i], &bt[i], &pr[i]); }
   while(done < n) {
      int idx = -1, min_pr = 1e9;
      for(int i=0; i<n; i++)
         if(at[i] <= cur && !vis[i] && pr[i] < min_pr) { min_pr = pr[i]; idx = i; }
      if(idx == -1) cur++;
      else { cur += bt[idx]; ct[idx] = cur; vis[idx] = 1; done++; }
   }
   printf("\nProcess\tAT\tBT\tPr\tCT\tWT\tTAT\n");
   for(int i=0; i<n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt/n, ttat/n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 2

Process  AT       BT       CT       WT       TAT
P1       0        8        14       6        14
P2       1        4        7        2        6
P3       2        2        4        0        2

Average Waiting Time = 2.67
Average Turnaround Time = 7.33
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 9
Enter Arrival Time and Burst Time for P4: 3 5

Process  AT       BT       CT       WT       TAT
P1       0        8        17       9        17
P2       1        4        5        0        4
P3       2        9        26       15       24
P4       3        5        10       2        7

Average Waiting Time = 6.50
Average Turnaround Time = 13.00
```

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process  AT       BT       CT       WT       TAT
P1       0        24       30       6        30
P2       0        3        3        0        3
P3       0        3        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

B. preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, at[10], bt[10], pr[10], rt[10], ct[10], done = 0, time = 0, min_pr, idx;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i + 1);
      scanf("%d%d%d", &at[i], &bt[i], &pr[i]); rt[i] = bt[i];
   }
   while (done < n) {
      min_pr = 1e9; idx = -1;
      for (int i = 0; i < n; i++)
         if (at[i] <= time && rt[i] > 0 && pr[i] < min_pr) { min_pr = pr[i]; idx = i; }
      if (idx == -1) { time++; continue; }
      rt[idx]--; time++;
      if (rt[idx] == 0) { done++; ct[idx] = time; }
   }
   printf("\nProcess\tAT\tBT\tPr\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases :

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       24      3       30      6       30
P2      0       3       1       3       0       3
P3      0       3       2       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 7 3
Enter Arrival Time, Burst Time and Priority for P2: 1 4 1
Enter Arrival Time, Burst Time and Priority for P3: 2 1 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       7       3       12      5       12
P2      1       4       1       5       0       4
P3      2       1       2       6       3       4

Average Waiting Time = 2.67
Average Turnaround Time = 6.67
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority for P1: 0 10 3
Enter Arrival Time, Burst Time and Priority for P2: 1 1  1
Enter Arrival Time, Burst Time and Priority for P3: 2 2 4
Enter Arrival Time, Burst Time and Priority for P4: 3 1 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       10      3       12      2       12
P2      1       1       1       2       0       1
P3      2       2       4       14      10      12
P4      3       1       2       4       0       1

Average Waiting Time = 3.00
Average Turnaround Time = 6.50
```

Aim: write a c-program for Round Robin scheduling algorithm A. Time Quantum=2, B. Time Quantum=10

A. Time Quantum=2

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Time Quantum (TQ) = 2 and Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the first process from the Ready Queue.
    - Execute the process for 2 units or until it finishes.
    - Update RT and time.
    - Add any newly arrived processes to the Ready Queue.
    - If RT > 0, add the process back to the Ready Queue; otherwise, calculate CT and mark it as completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

B. Time Quantum=10

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Time Quantum (TQ) = 10 and Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the first process from the Ready Queue.
    - Execute the process for 10 units or until it finishes.
    - Update RT and time.
    - Add any newly arrived processes to the Ready Queue.
    - If RT > 0, add the process back to the Ready Queue; otherwise, calculate CT and mark it as completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

A. Time Quantum=2

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, tq = 2, at[10], bt[10], rt[10], ct[10], done = 0, time = 0;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];}
   while (done < n) {
      int exec = 0;
      for (int i = 0; i < n; i++) {
         if (at[i] <= time && rt[i] > 0) {
            exec = 1;
            int slice = (rt[i] > tq) ? tq : rt[i];
            rt[i] -= slice; time += slice;
            if (rt[i] == 0) { done++; ct[i] = time; } }}
      if (!exec) time++;}
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat); }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 1

Process AT      BT      CT      WT      TAT
P1      0       5       9       4       9
P2      1       3       8       4       7
P3      2       1       5       2       3

Average Waiting Time = 3.33
Average Turnaround Time = 6.33
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 2
Enter Arrival Time and Burst Time for P4: 4 1

Process AT      BT      CT      WT      TAT
P1      0       5       12      7       12
P2      1       4       11      6       10
P3      2       2       6       2       4
P4      4       1       7       2       3

Average Waiting Time = 4.25
Average Turnaround Time = 7.25
```

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 3 4
Enter Arrival Time and Burst Time for P2: 5 6
Enter Arrival Time and Burst Time for P3: 7 9

Process AT      BT      CT      WT      TAT
P1      3       4       11      4       8
P2      5       6       17      6       12
P3      7       9       22      6       15

Average Waiting Time = 5.33
Average Turnaround Time = 11.67
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

B. Time Quantum=10

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, tq = 10, at[10], bt[10], rt[10], ct[10], done = 0, time = 0;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];
   }
   while (done < n) {
      int exec = 0;
      for (int i = 0; i < n; i++) {
         if (at[i] <= time && rt[i] > 0) {
            exec = 1;
            int slice = (rt[i] > tq) ? tq : rt[i];
            rt[i] -= slice; time += slice;
            if (rt[i] == 0) { done++; ct[i] = time; }
         }
      }
      if (!exec) time++;
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 25
Enter Arrival Time and Burst Time for P2: 2 6
Enter Arrival Time and Burst Time for P3: 4 12

Process AT      BT      CT      WT      TAT
P1      0       25      43      18      43
P2      2       6       16      8       14
P3      4       12      38      22      34

Average Waiting Time = 16.00
Average Turnaround Time = 30.33
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 15
Enter Arrival Time and Burst Time for P2: 1 8
Enter Arrival Time and Burst Time for P3: 3 22
Enter Arrival Time and Burst Time for P4: 5 5

Process AT      BT      CT      WT      TAT
P1      0       15      38      23      38
P2      1       8       18      9       17
P3      3       22      50      25      47
P4      5       5       33      23      28

Average Waiting Time = 20.00
Average Turnaround Time = 32.50
```

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 24

Process AT      BT      CT      WT      TAT
P1      0       3       3       0       3
P2      0       3       6       3       6
P3      0       24      30      6       30

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Name of the Experiment:

Experiment No: **Date:**

25071A6226 23

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: Banker's algorithm and Analysis of the scheduling algorithms

Experiment No: 3 **Date:** 23-07-2026

Aim: Make the analysis of all scheduling algorithms

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Burst Time for P1: 24
Enter Burst Time for P2: 3
Enter Burst Time for P3: 3

Process    BT        WT          TAT
P1         24        0           24
P2         3         24          27
P3         3         27          30

Avg WT = 17.00  |  Avg TAT = 27.00
```

FCFS scheduling without arrival times

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      24      0       24
P2      0       3       27      24      27
P3      0       3       30      27      30

Average Waiting Time = 17.00
Average Turnaround Time = 27.00
```

FCFS scheduling arrival times

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       3       0       3
P3      0       3       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

SJF scheduling non-preemptive

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       3       0       3
P3      0       3       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

SJF scheduling preemptive

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       24      3       30      6       30
P2      0       3       1       3       0       3
P3      0       3       2       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Priority scheduling non-preemptive

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       24      3       30      6       30
P2      0       3       1       3       0       3
P3      0       3       2       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Priority scheduling preemptive

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       3       0       3
P3      0       3       10      7       10

Average Waiting Time = 6.33
Average Turnaround Time = 16.33
```

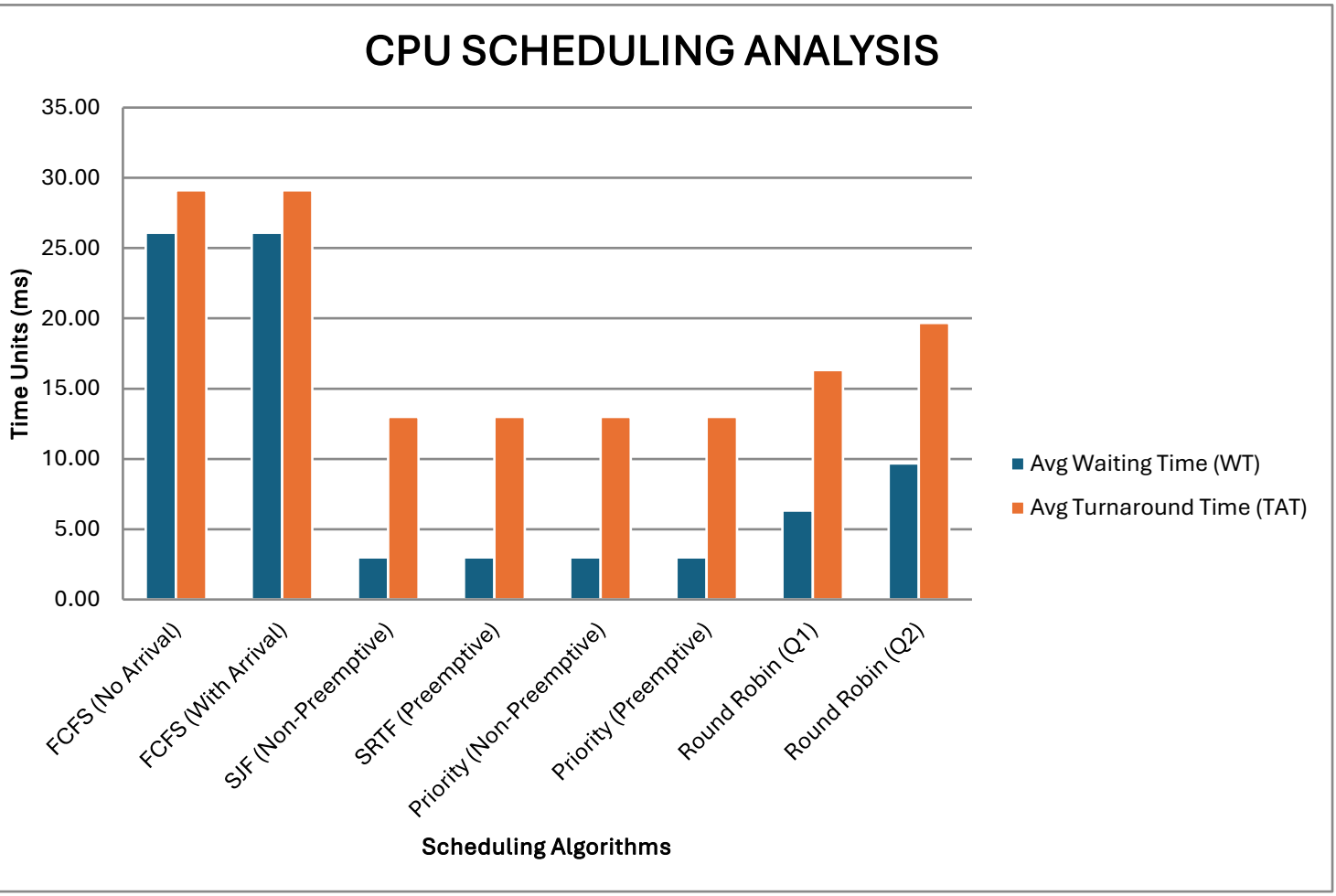
Round Robin scheduling, Time quantum=2

```
Roll no: 25071A6226
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       13      10      13
P3      0       3       16      16      16

Average Waiting Time = 9.67
Average Turnaround Time = 19.67
```

Round Robin scheduling, Time quantum=10


CPU SCHEDULING ANALYSIS
35.00
30.00
25.00
20.00
15.00
10.00
5.00
0.00
Time Units (ms)
FCFS (No Arrival)
FCFS (With Arrival)
SJF (Non-Preemptive)
SRTF (Preemptive)
Priority (Non-Preemptive)
Priority (Preemptive)
Round Robin (Q1)
Round Robin (Q2)
Scheduling Algorithms
Avg Waiting Time (WT)
Avg Turnaround Time (TAT)

Aim: write a c-program for Banker's algorithm

Algorithm:

1. **Start.**
2. Input the number of processes (n), resource types (m), Allocation, Max, and Available matrices.
3. Calculate the Need Matrix:
    - Need = Max − Allocation
4. Initialize:
    - Work = Available
    - Mark all processes as unfinished.
5. Repeat until all processes are finished:
    - Select an unfinished process whose Need ≤ Work.
    - If found:
        - Work = Work + Allocation
        - Mark the process as finished.
        - Add it to the Safe Sequence.
    - If no suitable process is found, stop.
6. Check the result:
    - If all processes are finished, the system is in a Safe State and display the Safe Sequence.
    - Otherwise, the system is in an Unsafe State.
7. **Stop**.

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6226\n");
   int n, m, alloc[10][10], max[10][10], need[10][10], avail[10], finish[10] = {0}, safeSeq[10], count = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter number of resource types: "); scanf("%d", &m);
   printf("Enter Allocation Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
   printf("Enter Max Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++) {
         scanf("%d", &max[i][j]);
         need[i][j] = max[i][j] - alloc[i][j];
      }
   printf("Enter Available Resources:\n");
   for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
   int work[10];
   for (int j = 0; j < m; j++) work[j] = avail[j];
  while (count < n) {
      int found = 0;
      for (int i = 0; i < n; i++) {
         if (!finish[i]) {
            int possible = 1;
            for (int j = 0; j < m; j++) {
               if (need[i][j] > work[j]) { possible = 0; break; }
            }
            if (possible) {
               for (int j = 0; j < m; j++) work[j] += alloc[i][j];
               safeSeq[count++] = i;
```

```
            finish[i] = 1;
            found = 1; }
        }
      }
      if (!found) break;
   }
   if (count == n) {
      printf("\nSystem is in a SAFE STATE.\nSafe Sequence: ");
      for (int i = 0; i < n; i++) printf("P%d ", safeSeq[i] + 1);
      printf("\n");
   } else {
      printf("\nSystem is in an UNSAFE STATE (Deadlock possible).\n");
   }
   return 0;}
```

Testcases:

```
Roll no: 25071A6226
Enter number of processes: 3
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
Enter Available Resources:
3 3 2

System is in an UNSAFE STATE (Deadlock possible).
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
Enter Available Resources:
3 3 2

System is in a SAFE STATE.
Safe Sequence: P2 P4 P1 P3
```

```
Roll no: 25071A6226
Enter number of processes: 4
Enter number of resource types: 3
Enter Allocation Matrix:
1 0 1
2 1 1
0 1 3
3 2 1
Enter Max Matrix:
5 4 4
4 3 3
2 2 5
4 3 3
Enter Available Resources:
1 1 0

System is in an UNSAFE STATE (Deadlock possible).
```